

Reinforcement Learning

Intuition & Logic

Reinforcement Learning (RL) is a type of Machine Learning where an **agent** learns to make decisions by interacting with an **environment**. Unlike supervised learning, there are no labelled examples — the agent learns through **trial and error**, receiving **rewards** for good actions and penalties for bad ones.

The core idea: *maximize cumulative reward over time*.

Real-World Applications

1. Robots learning to walk
2. Self-driving cars
3. Game-playing AI (Chess, Go)

Historical Milestones

1. **2013** — **DeepMind** combined Reinforcement Learning (RL) with Deep Learning (DL) → produced the **Deep Q-Network (DQN)**, enabling autonomous game playing.
2. **2016** — **AlphaGo** (Go game) defeated world champion **Lee Sedol**, showcasing the power of RL combined with neural networks.

Agent–Environment Interaction

The agent continuously interacts with the environment in a feedback loop:

Environment

State (S) $\uparrow\downarrow$ Action (A) | Reward (R) feedback: +ve or -ve

Agent (Learner)

The agent's **ultimate goal** is to **maximize cumulative rewards** through interaction with the environment.

Reinforcement Learning

Components

1 Action (A_t)

The set of all possible moves the agent can make at each time step. In a grid world: up (0), down (1), left (2), right (3).

2 Reward (R_t)

Feedback signal from the environment. Can be **positive (+ve)** for good actions or **negative (-ve)** as a penalty. The agent always tries to maximize total reward.

3 State (S_t)

A snapshot of the environment at time t . The state captures all information the agent needs to make a decision. Notation: $S \rightarrow$ snapshot of environment.

4 Policy (π)

The agent's strategy for choosing an action given a state. It is a mapping $S \rightarrow A$. Mathematically: $\pi(a|s)$ gives the probability of taking action a in state s .

5 Returns (G_t)

The total future reward (also called *gain*) from time step t onwards. Defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where γ (**gamma**) is the **discount factor**, $0 \leq \gamma \leq 1$.

• $\gamma = 0 \rightarrow$ only immediate returns matter • $\gamma = 1 \rightarrow$ long-term future rewards matter equally

6 Value Function (VF)

Estimates the expected return from a particular state s , following a policy π .

Type	Symbol	Meaning
State VF	$V_\pi(s)$	Expected return from state s , following policy π
Action VF	$Q_\pi(a,s)$	Expected return from state s , taking action a , following π
Optimal State VF	$V^*(s)$	Maximum possible return from state s across all policies
Optimal Action VF	$Q^*(a,s)$	Maximum possible return for action a in state s

Reinforcement Learning

Markov Decision Process (MDP)

MDP is the **mathematical model** underlying RL. It provides a formal framework for sequential decision-making. An episode/trajectory is represented as:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3 \dots$$

MDP tasks can be:

Type	Description	Example
Continuous	Infinite interaction — never terminates	Stock trading algorithms
Episodic	Finite interaction — terminates at goal	Chess / Go games (Episode)

Markov Property (rule): The next state depends only on the current state and current action — *not* on the entire past history.

$$S_{t+1} \leftarrow S_t, A_t$$

MDP Limitations:

- Not all real-world situations/environments fall neatly under MDP.
- Reward is represented as a single scalar value.

Grid World Example

A 3×4 grid where the agent starts at (2,0) and must reach the exit E at (2,3), while avoiding blocked cells X at (1,2) and (2,2).

Element	Definition
State (S)	Cell (i, j) → 2 numbers
Action (A)	0-3 (up, down, left, right) → 4 values
Reward (R)	Target cell E → 0 Non-E cells → -1
Terminate	Reach E OR 15 moves max

Q-Function & RL Algorithms

In **Model-Free RL**, the agent uses the **Action-Value Function (Q-function)** instead of the State-Value Function:

$$Q_{\pi}(s, a) \rightarrow \text{estimates the expected return for state } s \text{ and action } a \text{ under policy } \pi$$

$$Q_*(s, a) \rightarrow \text{picks the highest estimated return (optimal policy)}$$

Common RL algorithms:

1. **Q-Learning** — learns Q-values via Bellman equation (model-free, off-policy)
2. **SARSA** — State-Action-Reward-State-Action (on-policy)
3. **Policy Gradient Methods** — directly optimize the policy π using gradient ascent

Exploration–Exploitation Tradeoff

A fundamental challenge in RL: should the agent **explore** new possibilities or **exploit** the best-known action? Solution: ϵ -**Greedy Strategy**.

Strategy	Condition	Action Taken
Explore	if random value $r < \epsilon$	Choose a random action (new possibility)
Exploit	else ($r \geq \epsilon$)	Choose action with max Q-value (best known)

Key points about ϵ (epsilon):

- $0 \leq \epsilon \leq 1$ (e.g., $\epsilon = 0.1 \rightarrow 10\%$ chance to explore)
- ϵ **gradually decays** over time — agent explores more at the start and exploits more at the end.
- Start: explore + exploit $\rightarrow$ End: learn and apply best actions

Note: Feature scaling is important when using distance-based components in RL. Always ensure state representations are normalized for better convergence.

| Keep Learning & Keep Exploring!